

Estimação cega de ruído Gaussiano

Processamento Digital de Imagens

Docente: Prof. Dr. Cassio Dener Noronha Vinhal

Discente: Pedro Cézar Silva Ferreira

Motivação

"Como medir o ruído de uma imagem sem nunca ter visto a imagem limpa?"

Roteiro do Experimento

Parte A: Preparação

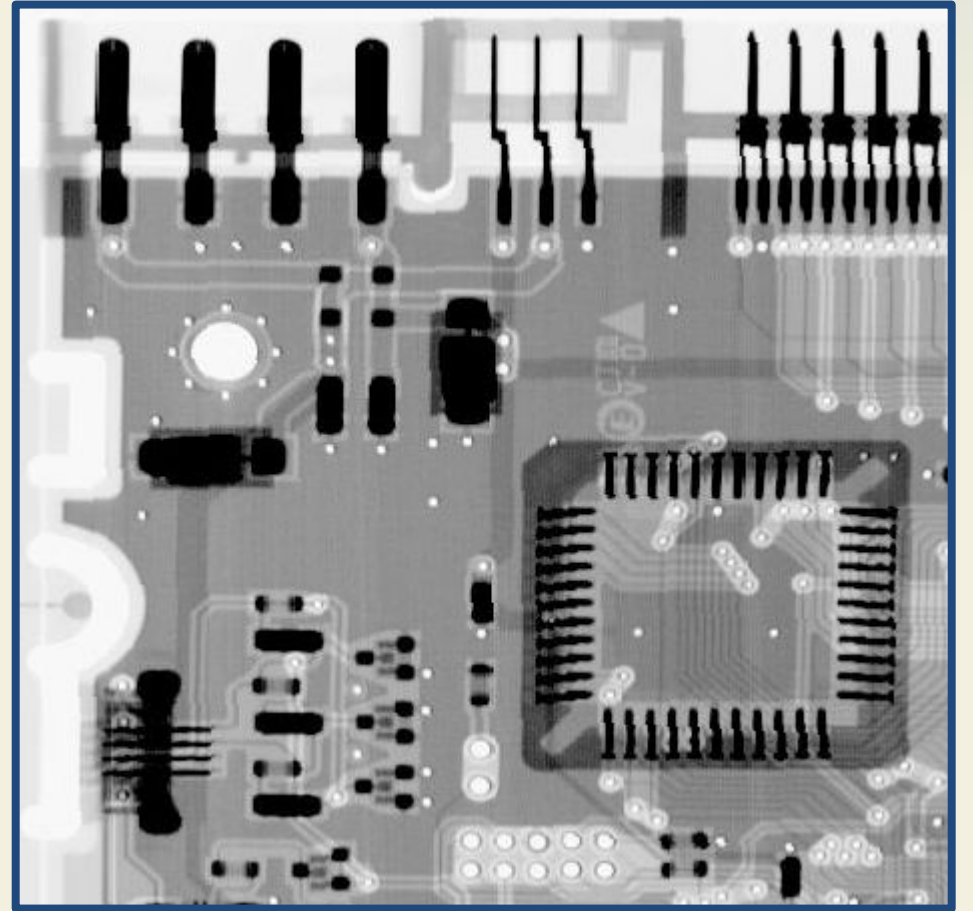
Parte B: A armadilha do desvio-padrão global

Parte C: Técnicas eficazes com diferenças e método MAD

Parte D: Teste de estresse utilizando 3 diferentes níveis de ruído

| Parte A: Preparação

```
Dimensões: (448, 464), dtype: float64  
Média ( $\mu_f$ ): 0.5912    Desvio-padrão ( $\sigma_f$ ): 0.2631  
Min/Max: 0.000 / 1.000
```



| Parte B: A armadilha do desvio-padrão global

O desvio-padrão global `np.std(g)` mede a variabilidade **total** da imagem, que inclui tanto o conteúdo real quanto o ruído. Por isso ele não serve para estimar σ do ruído.

| B.1: Adicionando ruído e comparando

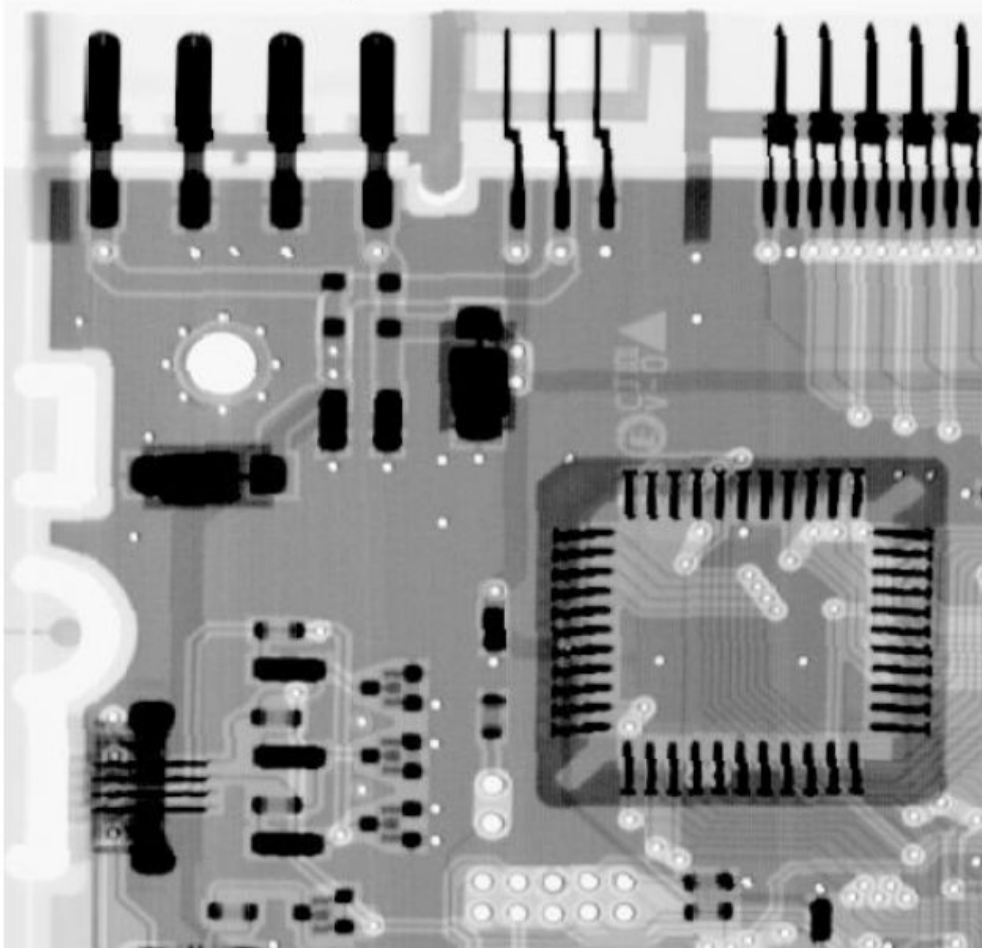
0.2644

Desvio-padrão global (g)

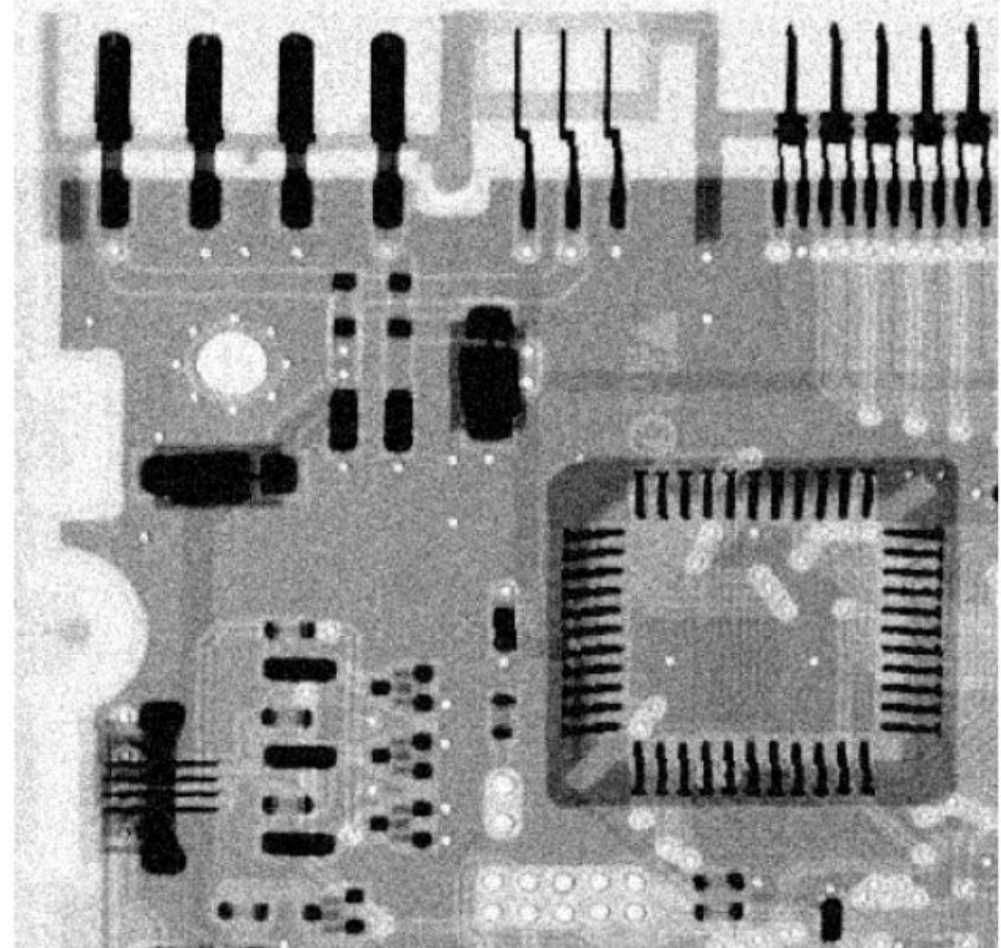
Ao injetar um ruído de $\sigma = 0.08$, o desvio-padrão global da imagem sobe apenas de 0.2631 para 0.2644.

B.2: Comparação Visual

Original (f) | std = 0.2631



Ruidosa (g) $\sigma_{\text{ruído}}=0.08$ | std = 0.2644



Parte C: Duas técnicas que funcionam

Técnica 1: Diferenças entre pixels vizinhos

Subtraindo pixels vizinhos, a estrutura lenta da imagem se cancela e sobra basicamente o ruído. Se o ruído tem desvio σ , a diferença de dois pixels vizinhos ruidosos tem desvio $\sqrt{2} \cdot \sigma$.

Técnica 2: MAD (Desvio Absoluto Mediano)

A mediana é robusta a outliers (bordas reais). Usamos:

$$\hat{\sigma}_{\text{MAD}} = \frac{\text{MAD}(d)}{0,6745 \cdot \sqrt{2}} \approx \frac{\text{MAD}(d)}{0,9539}$$

Os números 0,6745 e 0,9539 parecem mágicos mas têm origem clara: 0,6745 é a razão entre MAD e σ para a distribuição normal padrão (tabelado em qualquer livro de estatística), e $\sqrt{2}$ vem do mesmo raciocínio da diferença de gaussianas.

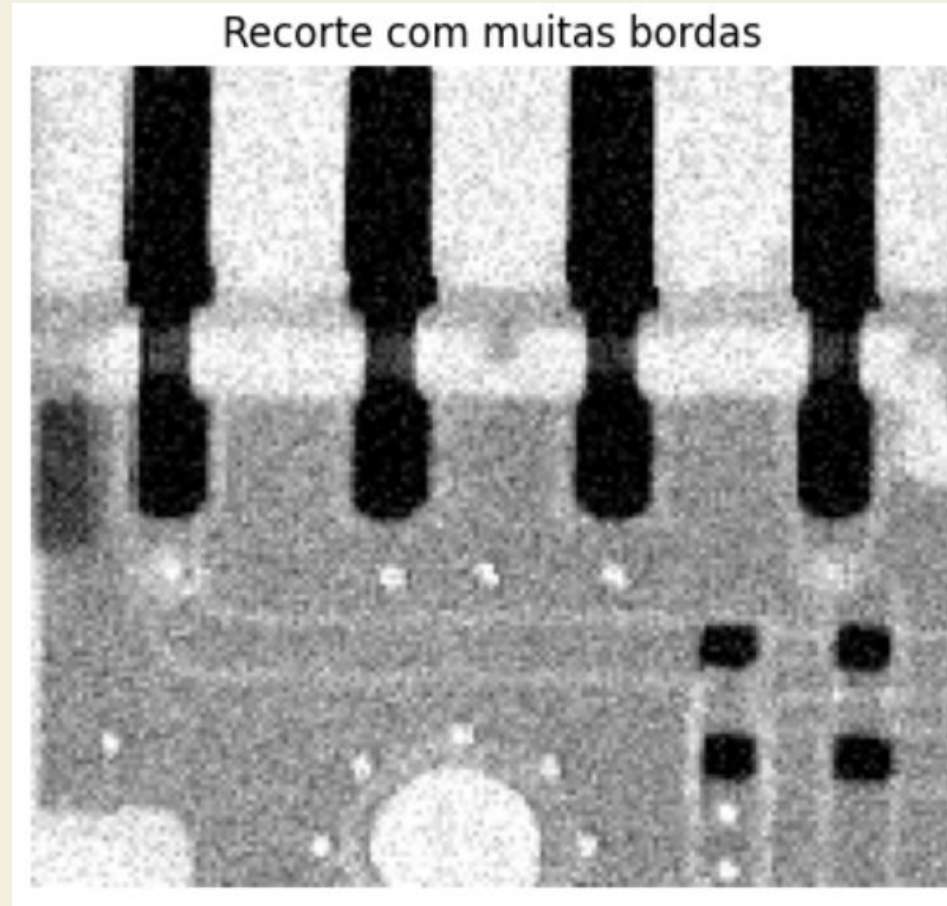
| C.1: Estimador por diferenças

```
def estimar_sigma_diff(img):  
    # calcula diferencas entre pixels vizinhos na horizontal  
    d = np.diff(img, axis=1)  
    # desvio-padrao das diferencas, corrigido pelo fator sqrt(2)  
    return d.std() / np.sqrt(2)  
  
# testa com a imagem inteira  
sigma_diff_total = estimar_sigma_diff(g)  
print(f'σ verdadeiro: {sigma_verdadeiro:.4f}')  
print(f'σ (diff) imagem inteira: {sigma_diff_total:.4f}')
```

σ verdadeiro:	0.0800
σ (diff) imagem inteira:	0.0972

| C.2: Recorte com muitas bordas

Recortamos uma região com muitos pinos/trilhas e vemos como o estimador por diferenças se comporta.



Em regiões com muitas bordas, o estimador diff superestima o ruído.

σ (diff) recorte pinos:

0.1023

| C.3: Estimador MAD

```
def estimar_sigma_mad(img):  
    # diferencas horizontais linearizadas  
    d = np.diff(img, axis=1).ravel()  
    # mad: mediana dos desvios absolutos em relacao a mediana  
    mad = np.median(np.abs(d - np.median(d)))  
    # converte mad para sigma usando fator 0.9539  
    return mad / 0.9539
```

σ verdadeiro:	0.0800
σ (diff) imagem inteira:	0.0972
σ (MAD) imagem inteira:	0.0795
σ (diff) recorte pinos:	0.1023
σ (MAD) recorte pinos:	0.0747

O MAD recupera o σ verdadeiro mesmo no recorte com muitos pinos.

É essa robustez que o torna o método padrão na prática.

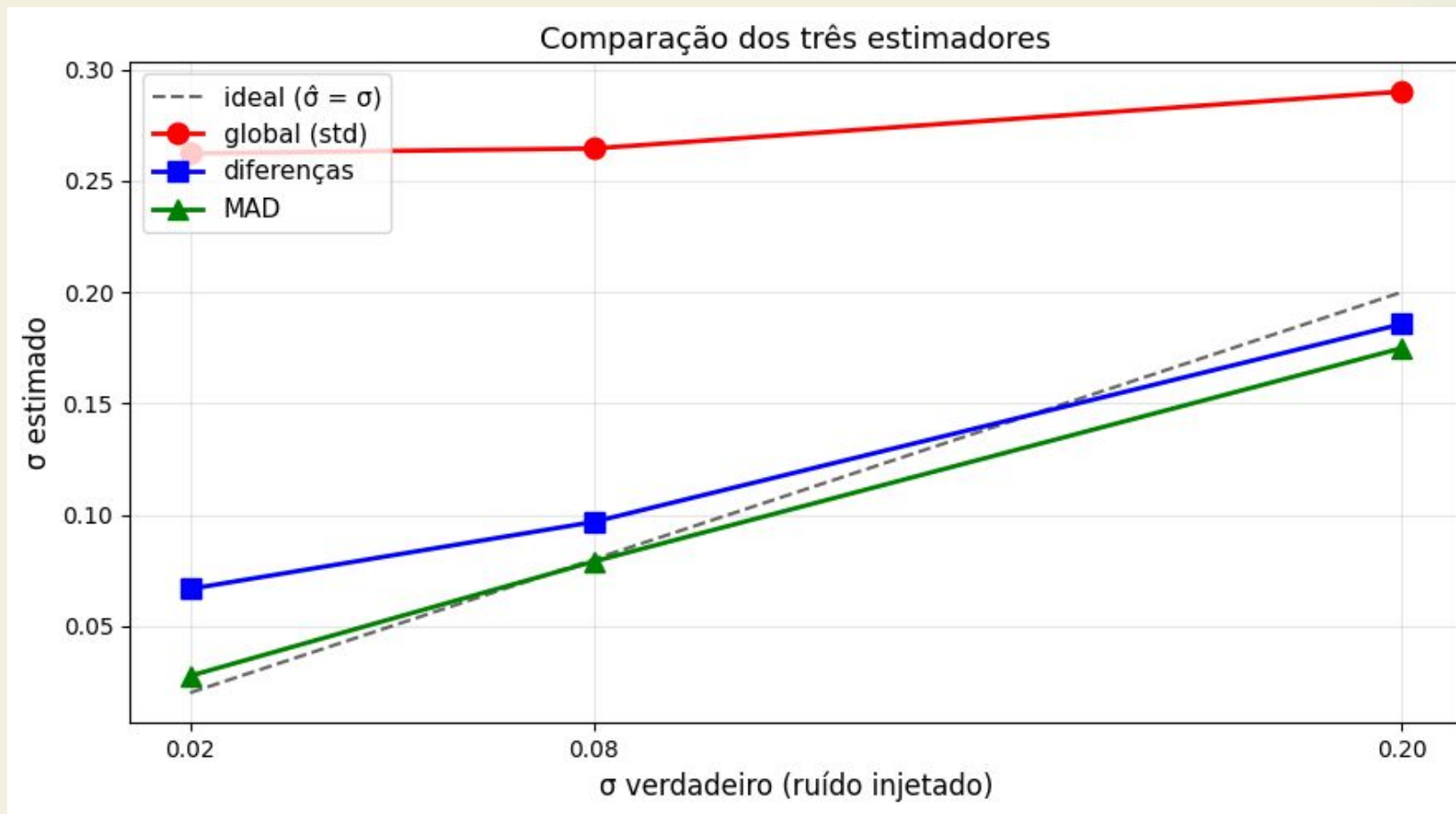
| D.1: Teste de estresse

Quantificamos a comparação variando o nível de ruído. Para cada σ na lista, geramos uma imagem ruidosa e aplicamos os três estimadores.

σ	verdadeiro	global (std)	diff	MAD	erro	global	erro	diff	erro	MAD
	0.02	0.2623	0.0667	0.0276		0.2423	0.0467	0.0076		
	0.08	0.2646	0.0968	0.0790		0.1846	0.0168	0.0010		
	0.20	0.2901	0.1857	0.1749		0.0901	0.0143	0.0251		

Observe: o erro absoluto do estimador global é sempre da ordem de $\sigma_f \approx 0,26$ (variabilidade natural da placa), independente do ruído real. Ele não vê o ruído, só vê a estrutura da imagem. Já diff e MAD acompanham o ruído verdadeiro de perto.

D.1: Teste de estresse



| D.1: O Efeito do Clip

Você deve ter notado uma curiosidade na linha de $\sigma = 0.20$: o MAD recupera ≈ 0.17 , e não 0.20. Isso não é falha do estimador, é uma consequência do que aplicamos para garantir que a imagem fique no intervalo válido.

A imagem da placa tem pixels já saturados em zero (o fundo preto) e algumas regiões claras próximas de 1. Com ruído forte, parte significativa das amostras de η tenta empurrar esses pixels para fora do intervalo $[0, 1]$, e o clip os trava nas bordas. Resultado: o ruído efetivamente presente na imagem g entregue ao estimador tem desvio menor que o σ que pedimos ao gerador.

Obrigado!

